ESIC

BTS Services Informatiques aux Organisations
Option SLAM – Solutions Logicielles et Applications Métiers

RAPPORT DE STAGE

Développement et fiabilisation de modules fonctionnels

au sein de l'application interne FenziConnect

PÉRIODE DU STAGE
Du 01/06/2026 au 01/08/2026

ENTREPRISE D'ACCUEIL
FenziConnect

PRÉSENTÉ PAR
Urielle Goma

MAÎTRE DE STAGE
Melissa Fenzi

TUTEUR PÉDAGOGIQUE
Chaabane Inoussa

Année scolaire 2025 – 2026

Remerciements

Je tiens à remercier l'ensemble de l'équipe de FenziConnect pour son accueil et l'accompagnement dont j'ai bénéficié tout au long de ce stage.

Je remercie également l'équipe pédagogique de l'ESIC pour le suivi assuré pendant la période de stage, ainsi que pour la formation dispensée dans le cadre du BTS SIO option SLAM, qui m'a permis d'aborder ce stage avec les bases techniques nécessaires.

Sommaire

Remerciements 2

1. Introduction 4

2. Présentation de l'entreprise 5

2.1 FenziConnect en bref 5

2.2 Organisation et service d'accueil 5

3. Contexte technique du stage 6

3.1 Architecture de l'application 6

3.2 Outils et méthodes de travail 6

4. Mission 1 – Fiabilisation du module Gestion des Congés 8

4.1 Contexte et objectif de la mission 8

4.2 Phase de diagnostic 8

4.3 Corrections apportées côté interface (Angular) 8

4.4 Construction de la partie serveur (Spring Boot) 8

4.5 Gestion des droits d'accès (Spring Security) 9

4.6 Ajout du formulaire de demande de congé 9

4.7 Intégration visuelle et tests 10

5. Mission 2 – Correction du formulaire de contact 11

5.1 Signalement et diagnostic 11

5.2 Résolution côté interface 11

5.3 Résolution côté serveur 11

6. Compétences mobilisées et acquises 12

7. Difficultés rencontrées et solutions apportées 13

8. Bilan personnel 14

9. Conclusion 15

Annexes 16

Glossaire 19

1. Introduction

Dans le cadre de ma formation en BTS Services Informatiques aux Organisations (SIO), option Solutions Logicielles et Applications Métiers (SLAM), à l'ESIC, j'ai effectué un stage en entreprise du 01 juin 2026 au 01 août 2026 au sein de la société FenziConnect.

Ce stage avait pour objectif de me confronter à un environnement de développement professionnel réel, en participant à la maintenance et à l'évolution d'une application déjà existante, développée selon une architecture découplée composée d'une interface web en Angular et d'une API back-end en Spring Boot.

Durant cette période, j'ai été chargée d'intervenir sur deux problématiques concrètes signalées par ma responsable : le dysfonctionnement du module de gestion des congés, et le bouton d'envoi non fonctionnel du formulaire de contact du site. Ces deux missions m'ont permis de mettre en pratique une démarche complète de développement logiciel : analyse et diagnostic d'un problème, recherche de la cause exacte, correction du code, création de nouvelles fonctionnalités, tests, puis restitution du travail réalisé.

Ce rapport présente dans un premier temps l'entreprise d'accueil et le contexte technique du stage, puis détaille les deux missions principales qui m'ont été confiées, avant de dresser un bilan des compétences acquises et des difficultés rencontrées.

2. Présentation de l'entreprise

2.1 FenziConnect en bref

FenziConnect est l'application interne sur laquelle porte mon stage : il s'agit d'une plateforme mettant en relation des clients et du personnel, permettant notamment la gestion de services, la gestion du personnel et des stagiaires, la facturation, le suivi de présence, ainsi que la gestion administrative des congés des collaborateurs.

2.2 Organisation et service d'accueil

J'ai intégré l'équipe de développement en charge de la maintenance et de l'évolution de l'application FenziConnect.

Mon rôle au sein de cette équipe consistait à intervenir sur des demandes de correction (bugs) et de développement de fonctionnalités, en autonomie encadrée, avec des points de suivi réguliers.

3. Contexte technique du stage

3.1 Architecture de l'application

L'application FenziConnect repose sur une architecture découplée, très répandue dans le développement web professionnel, qui sépare nettement la partie visible par l'utilisateur (le Front-End) de la partie qui gère les données et la logique métier (le Back-End).

Le Front-End (Angular) : il s'agit de l'interface avec laquelle l'utilisateur interagit dans son navigateur. Elle est développée avec le framework Angular, en composants dits « standalone » (autonomes), une approche moderne qui impose de déclarer explicitement, pour chaque écran, les outils dont il a besoin pour fonctionner.

Le Back-End (Spring Boot) : il s'agit du serveur qui reçoit les demandes envoyées par le Front-End, applique les règles de sécurité et de gestion, puis enregistre ou lit les informations dans la base de données. Il est développé en Java avec le framework Spring Boot, et sécurisé grâce au module Spring Security couplé à un système d'authentification par jeton (JWT).

La communication entre les deux parties : se fait via des requêtes HTTP (GET, POST, PUT, DELETE) envoyées par Angular vers des adresses (endpoints) exposées par des contrôleurs Java, au format REST. Chaque échange est encadré par une politique de sécurité CORS, qui autorise explicitement l'adresse du Front-End (http://localhost:4200) à dialoguer avec le serveur.

Cette organisation en couches m'a permis de bien comprendre, dès les premiers jours, la logique de circulation d'une information : de la saisie de l'utilisateur dans un formulaire jusqu'à son enregistrement définitif en base de données.

Couche	Technologies utilisées
Interface (Front-End)	Angular (composants standalone), TypeScript, HTML5, SCSS, Bootstrap
Serveur (Back-End)	Java, Spring Boot, Spring Data JPA, Spring Security
Sécurité / Authentification	JSON Web Token (JWT), configuration CORS
Base de données	Base relationnelle interrogée via JPA / Hibernate
Communication	API REST (HTTP GET, POST, PUT, DELETE), format JSON
Outils de développement	Visual Studio Code, IntelliJ IDEA, Git, Jira

3.2 Outils et méthodes de travail

Pour mener à bien mes missions, j'ai utilisé les outils suivants, en conditions professionnelles :

- Visual Studio Code pour le développement de la partie Angular (TypeScript, HTML, SCSS) ;
- IntelliJ IDEA pour le développement de la partie Java / Spring Boot ;
- Git et un dépôt distant (GitLab/GitHub) pour le versionnage du code, avec un travail organisé par branches de fonctionnalité (par exemple la branche FC-30 dédiée au module de gestion des congés) ;
- Jira pour le suivi des tickets et des demandes (analyses, bugs, développements) ;
- Les outils de développement du navigateur (console et onglet réseau) pour diagnostiquer les erreurs côté interface et identifier les requêtes HTTP en échec.

J'ai également suivi une méthode de travail structurée pour chaque mission : analyse du besoin ou du signalement, lecture et compréhension du code existant, diagnostic précis de la cause du problème, mise en œuvre de la correction ou du développement, tests fonctionnels, puis rédaction d'un compte rendu à destination de ma responsable.

4. Mission 1 – Fiabilisation du module Gestion des Congés

4.1 Contexte et objectif de la mission

Ma responsable m'a signalé que la partie « Gestion des Congés » de l'application ne fonctionnait pas correctement : la page ne s'affichait pas comme attendu et aucune information n'était visible côté utilisateur. Ma mission consistait à analyser en profondeur le code existant afin d'identifier l'origine exacte du problème, puis à corriger et compléter le module pour le rendre pleinement opérationnel, aussi bien côté interface que côté serveur.

4.2 Phase de diagnostic

La première étape a consisté à étudier le composant Angular chargé d'afficher la liste des congés (conge-list). Cette analyse a permis d'identifier plusieurs anomalies qui empêchaient le fonctionnement correct de la page :

- Un tableau d'importation vide : le composant, développé en mode « standalone », déclarait un tableau imports totalement vide dans sa configuration. Or, ce tableau doit obligatoirement contenir les outils Angular nécessaires (comme le CommonModule) pour que le HTML puisse utiliser des boucles d'affichage et des conditions. Sans cela, la page était incapable d'afficher la moindre donnée.
- Une incohérence de nommage : le nom de la classe du composant, dans le fichier principal, ne correspondait pas exactement au nom utilisé dans le fichier de tests automatiques associé. Ce désalignement provoquait des erreurs de compilation du projet.
- Une absence totale de lien de navigation : même si la page avait fonctionné correctement, aucun bouton ni aucun lien n'avait été prévu dans le menu principal de l'application pour permettre à un utilisateur d'y accéder.
- Un contenu HTML vide : le fichier censé afficher le tableau des congés ne contenait qu'un texte de test généré automatiquement par Angular lors de la création du composant, sans aucune structure réelle d'affichage.

Ce diagnostic a ensuite été formalisé dans un compte rendu écrit, remis à ma responsable, afin de valider avec elle le périmètre exact des corrections à apporter avant de démarrer le développement.

4.3 Corrections apportées côté interface (Angular)

Une fois le diagnostic validé, j'ai procédé aux corrections suivantes sur les fichiers du module :

- Ajout du CommonModule dans les imports du composant, afin de réactiver l'affichage dynamique de la liste, la gestion de l'état de chargement et les badges de couleur selon le statut de chaque congé (en attente, validé, refusé) ;
- Harmonisation du nom de la classe du composant sur l'ensemble des fichiers concernés, afin de faire disparaître les erreurs de compilation liées au conflit de nommage ;
- Construction complète du template HTML (conge-list.html) : un tableau présentant l'identifiant, le type de congé, les dates de début et de fin, le statut sous forme de badge coloré, ainsi que des boutons d'action (valider, refuser, supprimer) selon l'état de la demande, avec la gestion d'un indicateur de chargement et d'un message si la liste est vide ;
- Ajout d'un lien de navigation « Gestion Congés » dans le menu principal de l'application, en utilisant la directive routerLink d'Angular pour permettre un changement de page sans rechargement complet du navigateur.

Ce travail m'a permis de comprendre concrètement le fonctionnement des composants autonomes (standalone components) d'Angular, ainsi que l'importance de la déclaration explicite des dépendances d'un composant, contrairement à l'ancien système de modules globaux.

4.4 Construction de la partie serveur (Spring Boot)

Au cours de l'investigation, j'ai constaté qu'aucune partie serveur n'existait encore pour la gestion des congés : ni entité de base de données, ni service, ni contrôleur. J'ai donc construit l'ensemble de cette chaîne technique, en suivant l'architecture déjà utilisée pour les autres modules de l'application :

- Conge.java : l'entité représentant un congé en base de données, avec ses attributs (type, date de début, date de fin, statut) et ses méthodes d'accès (getters/setters) ;
- CongeRepository.java : l'interface permettant d'effectuer automatiquement les opérations de lecture et d'écriture en base de données, grâce à Spring Data JPA ;
- CongeService.java : la classe contenant la logique métier, avec les méthodes de listage, de validation, de refus, de suppression et de création d'une demande de congé, cette dernière forçant systématiquement le statut initial à « EN_ATTENTE » afin de garantir la cohérence des données ;
- CongeController.java : le contrôleur exposant les différentes routes de l'API REST (/api/conges) consommées par l'application Angular, avec les méthodes GET, PUT et DELETE correspondant aux actions de la liste des congés.

La création de ces quatre fichiers, en respectant scrupuleusement les conventions déjà en place dans le projet (nommage des packages, structure des dossiers), m'a permis de mieux comprendre le rôle de chacune des couches d'une application Spring Boot : entité, dépôt de données, service et contrôleur.

4.5 Gestion des droits d'accès (Spring Security)

Une fois la liaison entre Angular et l'API établie, une erreur 403 « Accès refusé » est apparue lors du chargement de la page. L'analyse du fichier de configuration de sécurité (SecurityConfig.java) a montré que la route /api/conges n'était pas déclarée parmi les routes autorisées, et tombait donc automatiquement sous la règle générale imposant une authentification pour toute autre requête.

J'ai ajouté la route des congés à la liste des routes accessibles, en cohérence avec les règles déjà appliquées aux autres fonctionnalités de l'application. Une fois cette correction effectuée, une nouvelle erreur (404, « ressource introuvable ») a confirmé que la sécurité laissait désormais bien passer la requête, mais qu'aucun contrôleur n'écoutait encore sur cette adresse — ce qui a été résolu par la création du CongeController décrite au point précédent.

Cette étape m'a permis d'appréhender concrètement le fonctionnement de Spring Security, la différence entre une erreur d'autorisation (403) et une erreur de routage (404), ainsi que l'importance capitale de la configuration CORS pour permettre à une application Front-End et une application Back-End, hébergées sur des ports différents, de communiquer en toute sécurité.

4.6 Ajout du formulaire de demande de congé

Une fois la liste des congés opérationnelle, ma responsable m'a demandé d'ajouter la fonctionnalité permettant à un utilisateur de soumettre une nouvelle demande de congé directement depuis l'application.

J'ai développé un nouveau composant demande-conges, comprenant :

- Un formulaire HTML (demande-conges.html) proposant un menu déroulant pour le type de congé et deux champs de date (début et fin), avec des boutons « Annuler » et « Soumettre la demande » ;
- Un contrôleur TypeScript (demande-conges.ts) chargé de récupérer les données saisies, d'appeler le service Angular d'envoi des données, puis de rediriger automatiquement l'utilisateur vers la liste des congés une fois la demande enregistrée avec succès ;
- Une évolution du service Angular (conge.service.ts) et du contrôleur Java (CongeController.java) pour transmettre, en plus des informations du congé, l'identifiant de l'utilisateur à l'origine de la demande.

J'ai ensuite ajouté un bouton « Nouvelle demande » sur la page de liste des congés ainsi qu'un lien dédié dans le menu latéral, tous deux pointant vers ce nouveau formulaire via le système de routage d'Angular.

4.7 Intégration visuelle et tests

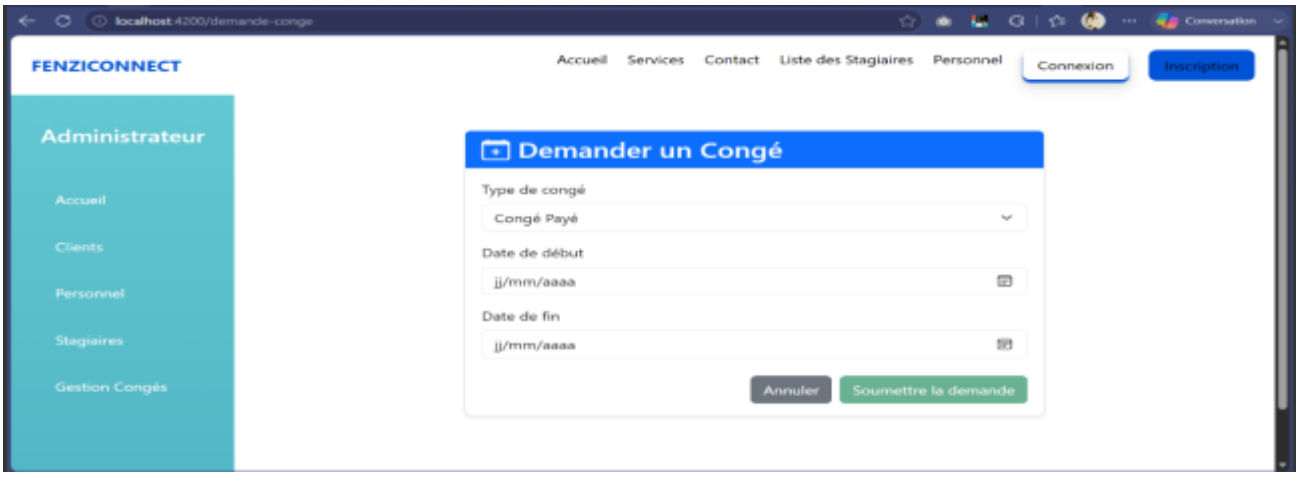
Lors de l'intégration du menu latéral (sidebar) commun à toute l'application sur la page de gestion des congés, celui-ci s'affichait initialement sur toute la largeur de l'écran, au lieu de se positionner sur le côté. J'ai corrigé cet affichage en structurant la page avec un conteneur flexible (classe Bootstrap d-flex), qui place correctement le menu latéral à gauche et le contenu principal à droite.

Enfin, j'ai réalisé une série de tests fonctionnels pour valider l'ensemble du parcours : navigation depuis le menu, affichage de la liste (y compris le cas d'une liste vide avec un message d'information), soumission d'une nouvelle demande de congé, ainsi que les actions de validation, de refus et de suppression. Ces tests ont confirmé le bon fonctionnement de la chaîne complète, de la saisie utilisateur jusqu'à l'enregistrement en base de données.

Récapitulatif des fichiers concernés

Domaine	Statut	Fichiers
Interface (Front-End)	Fichiers créés	demande-conges.html, demande-conges.ts
Interface (Front-End)	Fichiers modifiés	conge-list.html, conge-list.ts, app.routes.ts, conge.service.ts
Serveur (Back-End)	Fichiers créés	Conge.java, CongeRepository.java, CongeService.java, CongeController.java
Serveur (Back-End)	Fichiers modifiés	SecurityConfig.java





5. Mission 2 – Correction du formulaire de contact

5.1 Signalement et diagnostic

La seconde mission concernait un autre dysfonctionnement signalé via un ticket : le bouton « Envoyer le message » du formulaire de contact du site ne déclenchait aucune action. L'interface s'affichait pourtant correctement, avec les champs Nom, Email et Message visibles et fonctionnels.

L'analyse de la console de développement du navigateur a montré qu'aucune requête HTTP n'était envoyée lors du clic sur le bouton : seules les données saisies par l'utilisateur apparaissaient sous forme de journal (log) dans la console. L'examen du code source a permis d'en identifier la cause exacte : la structure HTML du formulaire était correctement reliée à une méthode onSubmit() dans le fichier TypeScript, mais cette méthode se contentait d'afficher les données dans la console, sans jamais faire appel à un service capable d'envoyer réellement ces informations vers le serveur.

5.2 Résolution côté interface

Pour corriger ce problème, j'ai créé un nouveau service Angular dédié, contact.service.ts, chargé d'utiliser le module HttpClient pour envoyer les données du formulaire via une requête POST vers l'adresse du serveur.

J'ai ensuite modifié le composant du formulaire de contact (contact.ts) pour y injecter ce nouveau service, puis entièrement réécrit la méthode onSubmit() afin qu'elle appelle la méthode d'envoi du service et gère les deux cas de figure possibles à la réception de la réponse du serveur : un message de confirmation en cas de succès, ou un message d'erreur explicite en cas d'échec de l'envoi.

5.3 Résolution côté serveur

Les premiers tests ont révélé une seconde anomalie : l'adresse ciblée par le service Angular ne correspondait pas exactement à celle exposée par le contrôleur Spring Boot (MessageContactRest.java). J'ai corrigé cette adresse afin qu'elle pointe précisément vers l'URL attendue par le serveur.

Enfin, l'audit de la configuration d'envoi d'e-mails (paramètres SMTP dans le fichier application.properties) a mis en évidence un dernier point bloquant : le mot de passe utilisé pour l'authentification auprès du serveur de messagerie était une valeur générique de type « espace réservé », provoquant systématiquement une erreur d'authentification. Cette anomalie a été documentée dans mon rapport de correction, avec la recommandation de générer un mot de passe d'application dédié depuis le compte de messagerie sécurisé associé à l'entreprise, afin de finaliser la mise en production de cette fonctionnalité.

Récapitulatif des fichiers concernés

Domaine	Statut	Fichiers
Interface (Front-End)	Fichier créé	contact.service.ts
Interface (Front-End)	Fichier modifié	contact.ts
Serveur (Back-End)	Fichiers vérifiés	MessageContactRest.java, application.properties (configuration SMTP)

6. Compétences mobilisées et acquises

Ce stage m'a permis de mobiliser et de renforcer un ensemble de compétences techniques et transversales, en cohérence avec le référentiel du BTS SIO option SLAM :

Compétences techniques

- Développement front-end avec Angular : composants standalone, directives structurelles (*ngIf, *ngFor), liaison de données, routage (RouterLink, RouterModule), formulaires pilotés par template (ngModel) ;
- Développement back-end avec Spring Boot : création d'entités JPA, de dépôts de données (repository), de services métier et de contrôleurs REST ;
- Sécurisation d'une API avec Spring Security : gestion des autorisations par route, compréhension du fonctionnement de l'authentification par jeton JWT et de la configuration CORS ;
- Consommation d'API REST côté client avec HttpClient et le module RxJS (Observable, subscribe) ;
- Mise en forme d'interfaces avec Bootstrap, notamment la construction de mises en page flexibles (Flexbox) ;
- Utilisation des outils de diagnostic du navigateur (console, onglet réseau) pour localiser précisément l'origine d'un dysfonctionnement ;
- Utilisation de Git pour le versionnage du code et le travail par branche de fonctionnalité.

Compétences transversales

- Analyse méthodique d'un problème technique, de la lecture du signalement jusqu'à l'identification de la cause racine ;
- Rédaction de comptes rendus techniques clairs, adaptés à des interlocuteurs non-techniques (responsable) comme techniques (documentation interne) ;
- Autonomie dans la recherche de solutions, tout en sachant solliciter un accompagnement lorsque nécessaire ;
- Organisation du travail à partir d'un ticket ou d'une demande, et respect des conventions déjà en place dans un projet existant.

7. Difficultés rencontrées et solutions apportées

Comme tout projet de développement, les missions confiées n'ont pas été exemptes de difficultés. Voici les principales rencontrées, ainsi que la manière dont j'y ai fait face :

La découverte d'un projet existant

Intervenir sur une application déjà développée par plusieurs personnes a nécessité un temps d'adaptation pour comprendre l'organisation des dossiers, les conventions de nommage et les choix techniques déjà réalisés (composants standalone, structure des routes, architecture des services). J'ai pris le temps de lire le code existant et de comparer les modules déjà fonctionnels avant de développer les miens, afin de rester cohérente avec l'ensemble du projet.

Des erreurs en cascade lors du diagnostic

La résolution du module de gestion des congés a nécessité plusieurs itérations : une première erreur corrigée en révélait souvent une seconde (passage d'une erreur de compilation, à une erreur 403, puis à une erreur 404). J'ai appris à traiter chaque erreur une à une, dans l'ordre où elle se présentait, plutôt que de chercher à tout corriger simultanément, ce qui m'a permis d'avancer de manière méthodique et de bien comprendre la cause de chaque blocage.

La compréhension du typage strict de TypeScript

Angular reposant sur TypeScript, certaines incohérences entre les modèles de données définis côté client et les objets réellement renvoyés par le serveur ont provoqué des erreurs de compilation. J'ai dû adapter ponctuellement le typage de certaines variables pour fluidifier les échanges, tout en gardant à l'esprit qu'un typage strict reste la bonne pratique à privilégier à terme.

La configuration de la sécurité de l'API

La configuration de Spring Security, avec la distinction entre routes publiques et routes protégées, a demandé une lecture attentive du fichier de configuration existant afin de ne pas fragiliser la sécurité des autres modules de l'application en ajoutant ma propre route.

8. Bilan personnel

Ce stage a représenté pour moi une première expérience concrète de développement en entreprise, sur une application réelle et déjà utilisée. Il m'a permis de sortir du cadre des exercices scolaires pour me confronter à des problématiques authentiques : un module qui ne fonctionne pas, un bouton sans effet, une base de données vide, une configuration de sécurité incomplète.

J'ai particulièrement apprécié la dimension d'investigation de mes missions : devoir remonter, à partir d'un simple signalement (« ça ne marche pas »), jusqu'à la ligne de code exacte à l'origine du problème. Cette démarche m'a appris à ne jamais me contenter d'une première hypothèse et à toujours vérifier mes corrections par des tests concrets.

Sur le plan technique, ce stage a considérablement renforcé ma maîtrise d'Angular et de Spring Boot, deux technologies au cœur du développement d'applications web modernes, ainsi que ma compréhension du fonctionnement global d'une architecture Front-End / Back-End sécurisée. Sur le plan humain, il m'a permis de développer mon autonomie, ma rigueur et ma capacité à communiquer un travail technique de manière claire, à travers la rédaction régulière de comptes rendus.

9. Conclusion

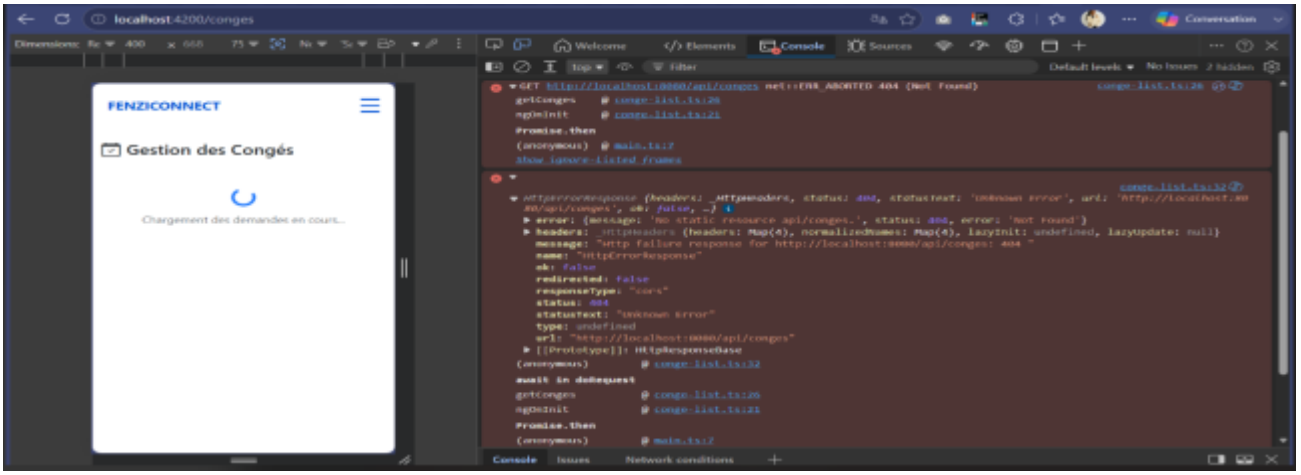
Ce stage de deux mois au sein de FenziConnect m'a permis de mettre en application, dans un contexte professionnel réel, les compétences développées durant ma formation en BTS SIO option SLAM. Les deux missions qui m'ont été confiées — la fiabilisation complète du module de gestion des congés et la correction du formulaire de contact — m'ont conduite à intervenir sur l'ensemble de la chaîne technique d'une application web moderne, de l'interface utilisateur jusqu'à la base de données, en passant par la sécurisation des échanges.

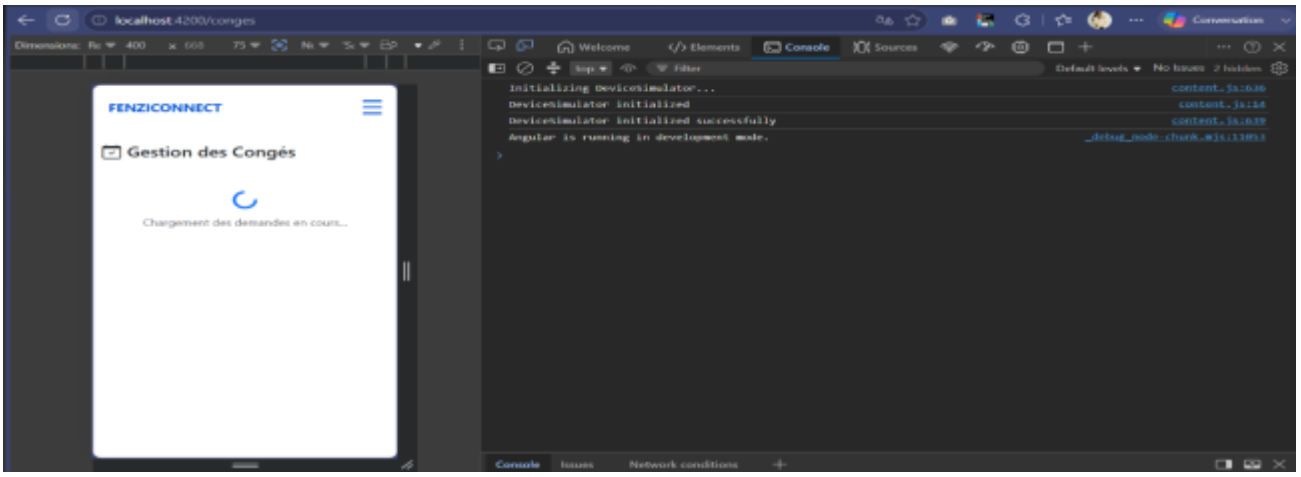
Au terme de cette expérience, le module de gestion des congés est pleinement fonctionnel : un utilisateur peut consulter la liste de ses demandes, en soumettre de nouvelles, et un gestionnaire peut les valider, les refuser ou les supprimer, le tout de manière sécurisée. Le formulaire de contact permet quant à lui l'envoi effectif des messages vers le serveur, la dernière étape restante concernant la finalisation de la configuration de messagerie électronique.

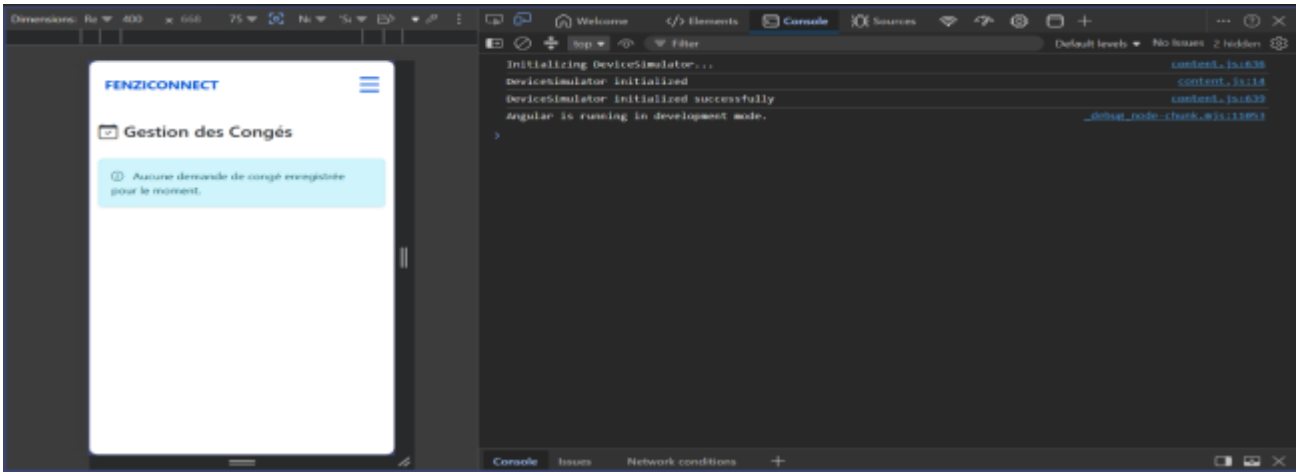
Cette expérience conforte mon intérêt pour le développement d'applications web et me donne des bases solides pour la suite de mon parcours, qu'il s'agisse de la poursuite de mes études ou de mon insertion professionnelle.

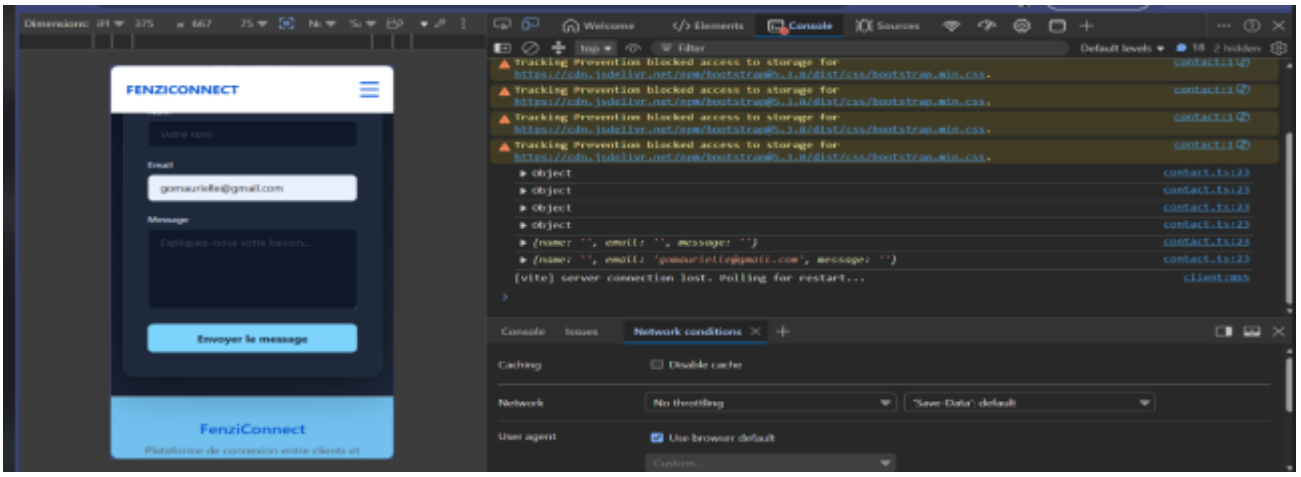
Annexes

Les captures d'écran suivantes illustrent le travail réalisé au cours du stage et complètent les éléments décrits dans ce rapport.

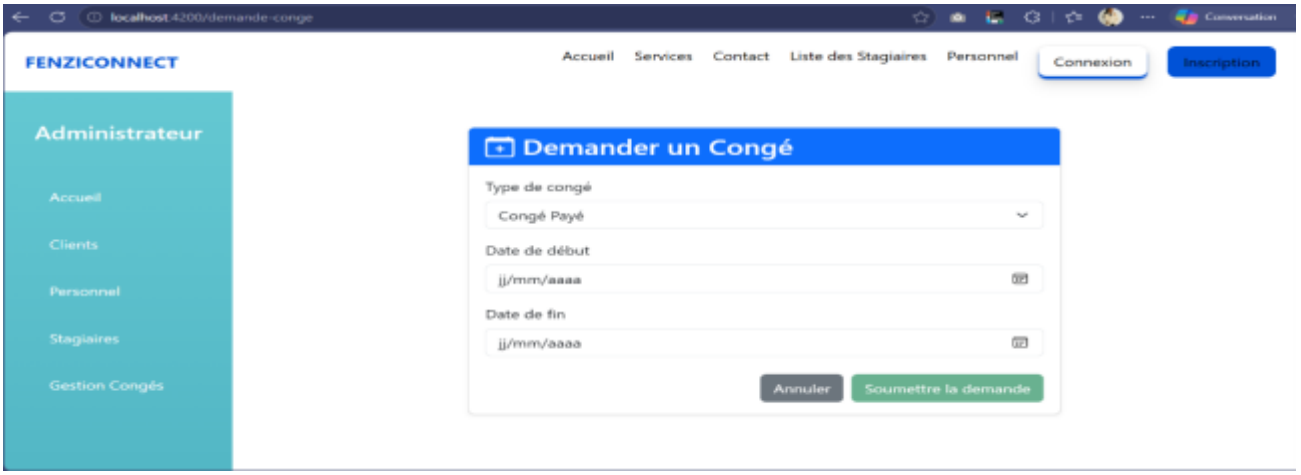







FENZICONNECT
Envoyer le message
FenziConnect


FENZICONNECT
Accueil
Services
Contact
Liste des Stagiaires
Personnel
Connexion
Inscription
Administrateur
Accueil
Clients
Personnel
Stagiaires
Gestion Congés
Gestion des Congés
Nouvelle demande
Aucune demande de congé enregistrée pour le moment.


localhost:4200/demande-conge
FENZICONNECT
Accueil
Services
Contact
Liste des Stagiaires
Personnel
Connexion
Inscription
Administrateur
Accueil
Clients
Personnel
Stagiaires
Gestion Congés
Demander un Congé
Type de congé
Congé Payé
Date de début
jj/mm/aaaa
Date de fin
jj/mm/aaaa
Annuler
Soumettre la demande

ATTESTATION DE STAGE

à remettre au stagiaire à l'issue du stage

ORGANISME D'ACCUEIL

FENZICONNECT

Adresse : 78 Avenue de suffren , Batiment B3, 75015 PARIS

Téléphone : 0750080372

Certifie que

LA STAGIAIRE

Urielle GOMA

Née le 16 février 2008

Adresse : 23 Rue Paul Bert, 92120 Montrouge

Téléphone : 0652957539

Courriel : gomaurielle08@gmail.com

ÉTUDIANT EN BTS SIO 01

AU SEIN DE ESIC

a effectué un stage prévu dans le cadre de ses études

DURÉE DU STAGE

Dates de début et de fin de stage : du 1 juin 2026 au 1 août 2026

Représentant une durée totale de 2 Mois

La durée totale du stage est appréciée en tenant compte de la présence effective du stagiaire dans l'organisme, sous réserve des droits à congés et autorisations d'absence prévus à l'article L.124-13 du code de l'éducation (art. L.124-18 du code de l'éducation). Chaque période au moins égale à 7 heures de présence consécutives ou non est considérée comme équivalente à un jour de stage et chaque période au moins égale à 22 jours de présence consécutifs ou non est considérée comme équivalente à un mois.

MONTANT DE LA GRATIFICATION VERSÉE AU STAGIAIRE

Le stagiaire a perçu une gratification de stage pour un montant total de 0.00 €

L'attestation de stage est indispensable pour pouvoir, sous réserve du versement d'une cotisation, faire prendre en compte le stage dans les droits à retraite. La législation sur les retraites (loi n°2014-40 du 20 janvier 2014) ouvre aux étudiants dont le stage a été gratifié la possibilité de faire valider celui-ci dans la limite de deux trimestres, sous réserve du versement d'une cotisation. La demande est à faire par l'étudiant dans les deux années suivant la fin du stage et sur présentation obligatoire de l'attestation de stage mentionnant la durée totale du stage et le montant total de la gratification perçue. Les informations précises sur la cotisation à verser et sur la procédure à suivre sont à demander auprès de la sécurité sociale.
(code de la sécurité sociale art. L.351-17 – code de l'éducation art.D.124-9)



PARIS, le 11 septembre 2026

Melissa FENZI, Responsable de FenziConnect

Glossaire

- API REST : interface permettant à deux applications de communiquer via des requêtes HTTP standardisées (GET, POST, PUT, DELETE).
- Composant standalone : composant Angular « autonome », qui déclare lui-même l'ensemble des outils dont il a besoin, sans dépendre d'un module global.
- CORS (Cross-Origin Resource Sharing) : mécanisme de sécurité du navigateur qui contrôle quelles origines (adresses) sont autorisées à dialoguer avec un serveur.
- Endpoint : adresse précise exposée par une API, associée à une action (par exemple /api/conges pour lister les congés).
- JWT (JSON Web Token) : jeton utilisé pour authentifier un utilisateur auprès d'un serveur sans avoir à renvoyer son mot de passe à chaque requête.
- ORM / JPA : technologie permettant de faire correspondre des objets Java à des tables d'une base de données relationnelle.
- Repository : interface permettant d'effectuer automatiquement les opérations de lecture et d'écriture en base de données.
- Routing / routage : mécanisme permettant de faire correspondre une adresse (URL) à un composant ou à un contrôleur précis.